

DEX Whale-Alert Bot

A production system that detects large wallet activity on decentralized exchanges in real time, filters it for genuine intent, delivers directional alerts to Telegram, and backtests every signal it produces.

Role: Solo — design, backend, infrastructure & analysis

Type: Real-time data pipeline + quant backtest

Status: Live, 24/7

[GitHub — source & reports](#)

[Live analytics \(JSON\)](#)

01 — OVERVIEW

Turning raw on-chain flow into filtered, measured signals

Large wallets often move a token on decentralized exchanges before the wider market reacts. This system watches for that activity end to end: it ranks the market by momentum, inspects the underlying DEX order flow to find wallets accumulating or distributing, confirms candidates against price and market-cap conditions, and delivers an alert — then logs every alert and replays its price path against a full risk-reward grid to measure whether the signal actually worked.

A two-stage funnel across a split runtime

- **Momentum universe** — live 5-minute / 30-minute / 24-hour leaderboards from Binance Futures decide which tokens are worth inspecting; CEX-only tokens are skipped and the next-ranked one backfills.
- **Wallet detection** — three signal types (sudden top-rank entry, step-up, sustained activity) run over a 60-minute window on both the buy and sell side.
- **Confirmation** — a candidate only fires when price agrees with its direction and either the wallet's volume clears 0.1% of market cap or price moves 3%+; a counter-move invalidates it.
- **Delivery** — Telegram alerts encode chain, direction, type and entry timeframe, and reconcile the executing wallet against the on-chain trade receipt to show the true token holder.
- **Backtest & analytics** — every alert is tracked and replayed against a 42-cell stop-loss / take-profit grid, queryable on demand.

The system is shaped by real constraints, not a whiteboard

INFRASTRUCTURE

The edge platform's IP got blocked by the DEX data API

Re-architected into a split runtime: detection moved to a VPS on an owned IP, while confirmation, alerting and state stayed on Cloudflare's edge — plus a KV price-relay so the edge never calls a rate-limited provider. Result: 24/7 operation at zero data-tier cost.

INCIDENT RESPONSE

A provider silently started requiring auth headers — every confirmation froze

Edge logs were empty, so I root-caused it with debug counters persisted into KV, identified the 403, fixed it the same day, and insulated the whole pipeline with the price-relay so a provider-side change can't freeze it again.

SIGNAL QUALITY

Most "whale activity" on the dominant chain was manufactured

Built an anti-wash filter that drops wallets trading both sides of a token in the same window, and reconciled the executing address against the trade receipt — surfacing the real holder (often a different wallet, a classic bot pattern) instead of an operator EOA showing a zero balance.

QUANT ANALYSIS

A self-serve backtest that finds the edge, not just reports a number

Each alert's price trail is replayed against every stop/take pair, then sliced by chain, signal type and entry timeframe. That analysis turned a breakeven aggregate into a clear direction: the edge is concentrated off the dominant chain and in one specific pattern — a finding no single headline number would reveal.

Signal-quality backtest over 1,199 signals

1,199

signals analyzed across 4 chains

+2.09 R

expectancy on the strongest
chain (Solana)

24/7

uptime — VPS + edge, ~\$4/mo

The backtest isolated where the system actually has an edge (specific chains and one signal pattern) versus where it merely produces noise, converting the raw stream into an actionable filtering strategy. *Note: this measures directional signal quality, not realised trading P&L.*

05 — STACK & SKILLS

What this project demonstrates

Node.js

Cloudflare Workers

Cloudflare KV

Linux / VPS · systemd

Telegram Bot API

EVM & Solana RPC

on-chain log parsing

Binance / GeckoTerminal / CoinGecko APIs

rate-limit engineering

quantitative backtesting

incident debugging

zero-dependency design